CSC241 – Object Oriented Programming (Java)

Theory Assignment 1

Object Oriented Analysis of a Smart Campus Allocation System

Name: Abdullah Bilal

Registration Number: SP26-BAI-026

Section: BAI

Instructor: Dr Muhammad Shahid Bhatti

Student-specific condition applied (last digit 6): Emergency continuity

Task 1 — Stakeholder and Requirement Analysis

Stakeholders and their needs

- Students — need fair, predictable access to spaces for individual study, group meetings and classes, without being consistently displaced by other users.
- Faculty members — need reliable priority for scheduled teaching sessions and urgent academic activity, so that classes are never left without a room.
- Administrators — need accurate utilisation data and the ability to set and adjust allocation policy across the whole campus.
- Security staff — need a complete, traceable record of who accessed which space and when, for safety and investigation purposes.
- Maintenance staff — need the authority to take a room out of service immediately when it becomes unsafe, overriding any existing bookings.
- Disability support staff / users with accessibility needs — need assurance that accessible rooms are usable when required, without their personal condition being disclosed to other users.

Three genuine requirement conflicts

Conflict 1 — Fair access vs faculty priority. Students expect fair access to shared spaces, but faculty members need priority for scheduled teaching and urgent academic activity. Granting faculty unconditional priority could make ordinary student access unpredictable; granting students strict equal priority could delay a class.

Assumption needed: *a precedence rule must be fixed before design — this assignment assumes that confirmed teaching reservations always outrank pending individual or group requests, while requests of the same priority level are ordered on a first-confirmed basis.*

Conflict 2 — High utilisation vs immediate safety action. The administration wants rooms booked as fully as possible, but maintenance staff must be able to block an unsafe room immediately, which can cancel or relocate an already-confirmed reservation and reduce utilisation.

Assumption needed: *safety is assumed to always override utilisation — a maintenance block takes effect instantly and unconditionally, and the resulting disruption (cancellations, relocations) is treated as an acceptable cost, handled through the same cancellation/relocation/notification pathway used for emergencies.*

Conflict 3 — Traceability vs privacy. Security staff require traceable access records tied to real identities, while users expect their personal information and the purpose of their bookings to remain private from other ordinary users.

Assumption needed: *a visibility boundary must be decided — this assignment assumes full identity and purpose are recorded internally for security and audit use only, while ordinary users see only whether a space is free or occupied, never who booked it or why.*

Facts vs assumptions

Facts explicitly stated in the scenario: the list of user categories, the four activity types and their differing rules, the existence of role changes during the semester, the existence of emergency building closures requiring cancellation/relocation/notification, and the explicit statement that fairness, recurring-reservation limits and simultaneous-request handling are undecided.

Assumptions introduced in this report (clearly separate from the above facts): the precedence rule between faculty and students, the decision that safety always overrides utilisation, the visibility boundary between internal audit data and what ordinary users can see, the specific reservation states and their transition rules, and the exact responsibility split between LearningSpace, Reservation and EmergencyEvent. These are design decisions made to resolve the conflicts above and are not given by COMSATS.

Task 2 — Object Oriented Conceptual Model

The model below proposes eleven classes (seven core classes plus four specialised LearningSpace subtypes) so that no single class is responsible for booking rules, space state, approvals, auditing and notification at the same time.

Class	Responsibility	Key attributes / operations
Person (abstract)	Represents any individual who interacts with the system; base identity and authentication.	personId, name, contactInfo; authenticate(), updateContactInfo()
Role	Represents one role a person currently holds (student, faculty, security, maintenance, administrator, disability support) and its permissions; supports multiple simultaneous and time-limited roles.	roleType, permissions, validFrom/validTo; isActive(), hasPermission()
LearningSpace (abstract)	Represents a bookable physical space and its real-time usability state.	spaceId, capacity, equipmentList, status; checkAvailability(), block(), unblock()
Classroom / Laboratory / SeminarRoom / ProjectSpace	Specialised subtypes of LearningSpace with type-specific attributes and rules (e.g. Laboratory adds a safety checklist).	hasProjector / safetyChecklist / hasVideoConf / isBookableLongTerm; runSafetyCheck() (Laboratory)
Reservation	Represents one booking request through its full lifecycle from request to completion, cancellation or relocation.	reservationId, startTime, endTime, status, purpose; confirm(), cancel(), relocate()
ActivityType	Represents the nature of the booking (individual, group meeting, class, academic event) and the capacity/approval rules that apply to it.	typeName, requiredCapacity, approvalRequired; validateAgainstSpace()
ApprovalWorkflow	Encapsulates the decision process for activity types that require approval, separating approval logic from the reservation itself.	approverId, decisionStatus, decisionTime; approve(), reject()
Equipment	Represents movable or fixed equipment that can be allocated to a reservation independently of the room.	equipmentId, name, movable, quantity; reserveUnit(), releaseUnit()
AuditRecord	Stores a traceable, security-relevant record of who did what and when, satisfying the traceability requirement without exposing this data to ordinary users.	recordId, action, actor, timestamp; createEntry()

Class	Responsibility	Key attributes / operations
Notification	Represents a message sent to an affected person, decoupled from the event that caused it.	message, channel, sentTime; send()
EmergencyEvent	Represents a building closure or similar disruption; orchestrates cancellation, relocation and notification across many reservations without performing the low-level work itself.	eventId, affectedBuilding, reason, status; triggerCancellations(), triggerRelocations(), notifyAffectedUsers()

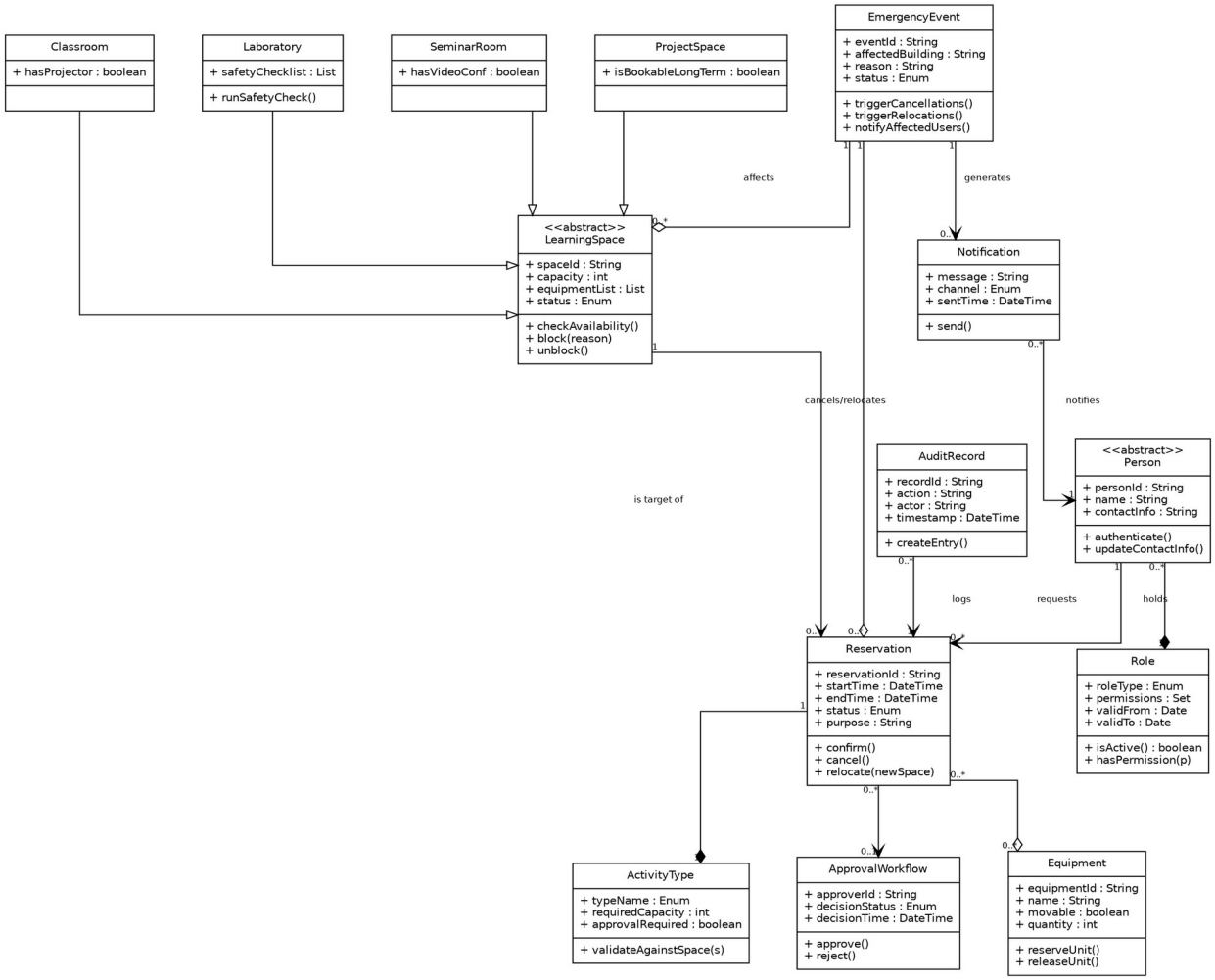


Figure 1. UML class diagram of the Smart Campus Learning Space Allocation System.

Separation of responsibilities

- LearningSpace and its subtypes only manage the physical space and its availability state — they know nothing about approval rules or notifications.
- Reservation only manages the lifecycle of one booking; it delegates approval decisions to ApprovalWorkflow and equipment allocation to Equipment, rather than deciding these itself.
- AuditRecord and Notification are separate from EmergencyEvent: EmergencyEvent orchestrates a building closure by calling Reservation.cancel()/relocate(), but the actual logging and messaging are performed by AuditRecord and Notification respectively, so EmergencyEvent does not become a single class that does everything.
- Role is kept separate from Person so that permission logic does not live inside the identity class, and so a person can hold several roles at once (justified further in Task 4).

Task 3 — Abstraction, Encapsulation and Invariants

Three selected classes and what they abstract

Reservation — abstracts the real-world act of booking a space for a time window, hiding the difference between an individual request, a group meeting, a class or an academic event behind a single lifecycle (see Task 5).

LearningSpace — abstracts a physical room together with its current real-time usability, hiding the underlying calendar of overlapping reservations behind a single checkAvailability() query.

EmergencyEvent — abstracts an external disruption, such as the building closure required by the emergency-continuity condition, hiding the sequence of cancellations, relocations and notifications behind three coordinated operations.

Information hidden and how it may change

- Reservation.status is private; it can only move forward through confirm(), cancel() or relocate() — no external class may set it directly, which prevents an invalid jump such as PENDING straight to RELOCATED.
- LearningSpace.status and its reservation calendar are private; they change only through block(), unblock() and the internal bookkeeping inside confirm(), so no two callers can create an overlapping booking by writing to the calendar directly.
- EmergencyEvent.status and the list of affected reservations are private; the only way to act on them is triggerCancellations(), triggerRelocations() and notifyAffectedUsers(), so a closure can never be applied to some reservations and silently skip others.

Invariants

- A Reservation's endTime must always be later than its startTime.
- A LearningSpace cannot have two CONFIRMED reservations with overlapping time windows (no double-booking).
- A Reservation cannot reach CONFIRMED if its ActivityType requires approval unless a matching ApprovalWorkflow has recorded an approve() decision.
- A LearningSpace whose status is BLOCKED cannot have any new reservation reach CONFIRMED against it until unblock() is called.
- Every cancellation or relocation caused by an EmergencyEvent must produce a matching AuditRecord and a Notification to the affected user — no state change may happen silently.

Because every field that participates in these rules is private and reachable only through the operations listed above, each operation can check the relevant invariant before applying a change. This makes it impossible for a caller anywhere else in the system to force the model into an invalid state, which a public, freely-settable status field would not prevent.

Task 4 — Alternative Designs and Decision

The scenario states that a person may hold more than one role and that roles and permissions may change during the semester. Two designs were compared for representing this.

Option A — Inheritance

Person is a superclass with subclasses such as Student, Faculty, SecurityStaff, MaintenanceStaff and Administrator, each defined at compile time by its class.

Option B — Composition (role objects, as used in Task 2)

Person stays a single class that holds a collection of Role objects, each carrying its own permissions and validity period, and roles can be added, removed or expired independently of the Person object itself.

Comparison

- Extensibility: Java's single inheritance means Option A cannot represent a person who is simultaneously a Student and a Teaching Assistant without an awkward workaround; Option B simply attaches a second Role object.
- Coupling: Option A couples a person's identity permanently to one class at creation time; Option B keeps Person independent of any specific role's behaviour.
- Duplicate behaviour across multiple roles: Option A would need duplicated or multiply-inherited logic for combined roles; Option B reuses the same Role class for every combination.
- Ease of changing permissions at runtime: Option A would require replacing or re-casting the Person object to change its type, which is impractical; Option B lets a Role's validTo be set or a new Role be added at runtime without touching the Person object's identity.

Decision

Option B (composition using Role objects) is chosen for this scenario. It directly satisfies the stated requirement that roles and permissions may change during the semester and that a person may hold more than one role at once — something Java's single-inheritance model cannot express cleanly. This decision is reflected in the Person–Role composition already shown in the Task 2 class diagram.

Task 5 — Behaviour and Edge Cases

Reservation life cycle

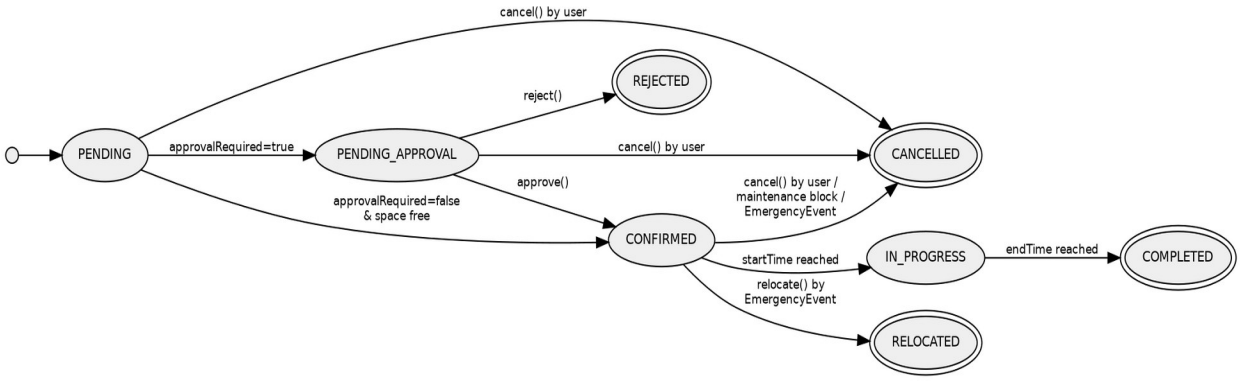


Figure 2. State diagram of the Reservation life cycle.

State	Meaning	Triggers / transitions
PENDING	Request submitted	approvalRequired = false and space free -> CONFIRMED; approvalRequired = true -> PENDING_APPROVAL; user cancels -> CANCELLED
PENDING_APPROVAL	ApprovalWorkflow created	approve() -> CONFIRMED; reject() -> REJECTED; user cancels -> CANCELLED
CONFIRMED	Space reserved for the time window	startTime reached -> IN_PROGRESS; cancel() (user, maintenance block, EmergencyEvent) -> CANCELLED; relocate() (EmergencyEvent) -> RELOCATED

State	Meaning	Triggers / transitions
IN_PROGRESS	Activity is currently taking place	endTime reached -> COMPLETED
COMPLETED / REJECTED / CANCELLED / RELOCATED	Terminal states	no further transitions; RELOCATED spawns a new Reservation in PENDING/CONFIRMED for the replacement space

Edge case 1 — Two simultaneous requests for the same space

When two requests target an overlapping time window on the same LearningSpace, both call confirm(), but LearningSpace owns the authoritative calendar and evaluates checkAvailability() for each request as it arrives. The first request that passes the check is CONFIRMED; the second fails the invariant check inside confirm() and is rejected with a reason, leaving it in PENDING for the user to choose another slot. LearningSpace enforces this decision, not Reservation, because only LearningSpace has full visibility of everything booked against it.

Edge case 2 — Maintenance block after confirmation

Maintenance staff call LearningSpace.block(reason), which the LearningSpace enforces on itself immediately and unconditionally, regardless of any existing CONFIRMED reservations. LearningSpace then notifies each affected Reservation, which transitions itself to CANCELLED or RELOCATED; AuditRecord logs the action and Notification informs each affected user. LearningSpace enforces the authority to block; Reservation enforces its own resulting state change.

Edge case 3 — Emergency cancellation or relocation (building closure)

A building closure is represented as an EmergencyEvent. It calls triggerCancellations() and/or triggerRelocations(), which iterate over every Reservation linked to the affected LearningSpace objects and call each Reservation's own cancel() or relocate() method — EmergencyEvent orchestrates the response but does not mutate reservation state directly. notifyAffectedUsers() then creates a Notification for every affected Person, and AuditRecord logs each change for later audit, satisfying the requirement that the system retain enough information for an audit and notify affected users.

Task 6 — Independent Exploration: Role-Based Access Control

Role-Based Access Control (RBAC) is an authorisation approach in which permissions are not granted to individual users directly but to named roles, and users are granted access simply by being assigned one or more of those roles. Instead of asking “what can this specific person do,” the system asks “what can this role do,” and a user's effective permissions become the union of the permissions of every role currently assigned to them. This indirection is what makes RBAC well suited to organisations where many people share the same responsibilities and where those responsibilities change over time, since only the role-to-permission mapping needs to be updated rather than every affected user account.

In the Smart Campus model, RBAC improves the Person–Role relationship from Task 2 and Task 4 directly: instead of hard-coding checks such as “is this person a faculty member,” each Role carries a permission set (for example, ApproveReservation, BlockRoom, ViewAuditLog), and operations such as LearningSpace.block() or ApprovalWorkflow.approve() simply verify hasPermission() on the caller's active roles. This lets a person who is both a student and a teaching assistant automatically gain teaching-assistant permissions without any change to the Person or Reservation classes, and it lets permissions be revoked instantly by expiring a Role, exactly matching the requirement that roles and permissions may change during the semester.

Sources:

1. D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” ACM Transactions on Information and System Security, vol. 4, no. 3, 2001. Available: https://csrc.nist.gov/projects/role-based-access-control

2. OWASP Foundation, “Access Control Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html

Generative AI Disclosure

A generative AI assistant (Claude, Anthropic) was used to help structure this report, draft the UML class and state diagrams, and phrase the written analysis, based on the requirements stated in the assignment brief. All design decisions, assumptions and the final content were reviewed and are the responsibility of the author, Abdullah Bilal (SP26-BAI-026).